

Comparators are used to compare two objects. In this challenge, you'll create a comparator and use it to sort an array. The `Player` class is provided in the editor below. It has two fields:

1. ***name***: a string.
2. ***score***: an integer.

Given an array of n `Player` objects, write a comparator that sorts them in order of decreasing score. If **2** or more players have the same score, sort those players alphabetically ascending by name. To do this, you must create a `Checker` class that implements the `Comparator` interface, then write an `int compare(Player a, Player b)` method implementing the `Comparator.compare(T o1, T o2)` method. In short, when sorting in ascending order, a comparator function returns **-1** if $a < b$, **0** if $a = b$, and **1** if $a > b$.

Declare a `Checker` class that implements the comparator method as described. It should sort first descending by score, then ascending by name. The code stub reads the input, creates a list of `Player` objects, uses your method to sort the data, and prints it out properly.

Example

$n = 3$ `data = [[Smith, 20], [Jones, 15], [Jones, 20]]`

Sort the list as `datasorted = [[Jones, 20], [Smith, 20], [Jones, 15]]`. Sort first descending by score, then ascending by name.

Input Format

The first line contains an integer, n , the number of players.

Each of the next n lines contains a player's ***name*** and ***score***, a string and an integer.

Constraints

- $0 \leq \text{score} \leq 1000$
- Two or more players can have the same name.
- Player names consist of lowercase English alphabetic letters.

Output Format

You are not responsible for printing any output to stdout. Locked stub code in Solution will instantiate a `Checker` object, use it to sort the `Player` array, and print each sorted element.

```

1  import java.util.*;
2
3  class Player {
4      String name;
5      int score;
6
7      Player(String name, int score) {
8          this.name = name;
9          this.score = score;
10     }
11 }
12 class Checker implements Comparator<Player> {
13     public int compare(Player a, Player b) {
14         if (a.score > b.score) return -1;
15         if (a.score < b.score) return 1;
16         return a.name.compareTo(b.name);
17     }
18 }
19
20
21 public class Solution {
22
23     public static void main(String[] args) {
24         Scanner scan = new Scanner(System.in);
25         int n = scan.nextInt();
26
27         Player[] player = new Player[n];
28         Checker checker = new Checker();
29
30         for(int i = 0; i < n; i++){
31             player[i] = new Player(scan.next(), scan.nextInt());
32         }
33         scan.close();
34
35         Arrays.sort(player, checker);
36         for(int i = 0; i < player.length; i++){
37             System.out.printf("%s %s\n", player[i].name, player[i].score);
38         }
39     }
40 }

```

Congratulations

You solved this challenge. Would you like to challenge your friends? [Twitter](#) [LinkedIn](#)

[Next Challenge](#)

✓ **Test case 0**

✓ Test case 1 [🔒](#)

✓ Test case 2 [🔒](#)

✓ Test case 3 [🔒](#)

✓ Test case 4 [🔒](#)

✓ Test case 5 [🔒](#)

✓ Test case 6

Compiler Message

Success

Input (stdin)

```
1 5
2 amy 100
3 david 100
4 heraldo 50
5 aakansha 75
6 aleksa 150
```

[Download](#)

Expected Output

[Download](#) ▼